

SHIVA ZEN

Sistema de Gestao para Clinica de Estetica

Jaqueline Aranha Estetica

Documento de Especificacao de Requisitos e Modelagem UML

Versao 2.1 | 21/06/2026

Sumario

Atualize este campo no Word (Ctrl+A, F9) para gerar o sumario.

1. Introducao

1.1 Objetivo

Este documento especifica os requisitos funcionais e nao-funcionais, as regras de negocio e a modelagem UML do sistema Shiva Zen — plataforma web de gestao para a clinica de estetica Jaqueline Aranha. O conteudo foi derivado diretamente do codigo-fonte da aplicacao (Django 5.2), garantindo aderencia ao comportamento real implementado.

1.2 Escopo

O sistema cobre o agendamento online publico (sem cadastro, identificado por telefone com verificacao OTP por SMS), o portal do profissional, o painel administrativo (agenda, clientes, prontuario, pacotes, promocoos, comissoes, NPS, notificacoes), alem de rotinas automaticas (lembretes, NPS, expiracao de pacotes, expurgo LGPD) e integracoes externas (SMS Zenvia, WhatsApp Meta, e-mail, Google Calendar, Cloudflare Turnstile).

1.3 Visao geral da arquitetura

Aplicacao Django monolitica servida por gunicorn, com processamento assincrono via Celery (worker + beat), PostgreSQL como banco primario, Redis (broker, cache e sessao) e frontend em Vite/Tailwind com PWA. A regra de negocio reside em uma camada de servicos e eventos de dominio (EventBus); o estado do atendimento e governado por uma maquina de estados (FSM). Detalhes na secao 8.

2. Atores do Sistema

Atores identificados a partir dos papéis de acesso e dos fluxos públicos/internos.

Ator	Descrição
Visitante / Cliente (público, sem login)	Pessoa que navega no site público e agenda online sem autenticação. Identidade real do booking é o TELEFONE. Acessa páginas públicas (home, serviços, equipe, promoções, depoimentos), agenda procedimento (3 steps), entra na lista de espera, e responde formulários públicos (anamnese pré-atendimento, pesquisa, NPS, assinatura de termo, confirmação de presença) via link mágico com token urlsafe. Deve ter ≥18 anos. Consentimentos LGPD (email marketing, WhatsApp NPS/confirmação) capturados com IP/timestamp.
Cliente Recorrente (cadastro existente)	Cliente já cadastrado (telefone OU email pertencem a um Cliente ativo). Para agendar/reagendar precisa passar pelo gate de OTP via SMS (Zenvia) — anti-sequestro de cadastro. Faz login sem senha em 'Meus Agendamentos' via OTP por email (código enviado por SMS ao telefone cadastrado), consulta agendamentos futuros/passados, reagenda (mín. 24h antecedência) e cancela via token seguro. Pode exercer direitos LGPD (ver/baixar dados, descadastrar marketing).
Profissional / Biomédica	Usuário vinculado a um Profissional ativo (decorator profissional_required). Acessa o Portal do Profissional: vê sua própria agenda do dia/semana, aprova ou rejeita agendamentos PENDENTES dele, marca atendimento como REALIZADO, adiciona anotações de sessão e consulta prontuário de clientes que já atendeu (ownership/anti-BOLA: 404 em atendimento de outro profissional). Pode ter Google Calendar conectado (feed ICS / pull).
Administrador / Gerente / Recepção (staff)	Usuário is_staff (decorator staff_required). Acesso total ao painel: dashboards (overview e financeiro com comissões), gestão de agendamentos (aprovar/rejeitar/cancelar/bulk/status AJAX), calendário drag-drop, clientes e prontuário, profissionais, procedimentos, preços, disponibilidade/exceções/bloqueios de agenda, pacotes (CRUD + venda), promoções (CRUD + disparo de email), lista de espera (notificar), termos de consentimento e compliance, anamneses (formulários), configurações do sistema, branding, usuários admin (CRUD + reset senha + ativar/desativar), auditoria (LogAuditoria), exportar relatório Excel. Login por email/senha em URL oculta com rate-limit; pode/deve ativar 2FA TOTP.
Sistema / Cron Externo (cron-job.org)	Ator automático. Endpoint POST /cron/run/<job>/ protegido por header X-Cron-Token (HMAC compare) dispara jobs Celery síncronos: lembrete D-1 (WhatsApp), NPS 24h pós-atendimento, alerta de detrator NPS por email, verificação/expiração de pacotes, aniversário de clientes, promoção mensal, limpeza de status (PENDENTE→CANCELADO e AGENDADO/CONFIRMADO→FALTOU após 24h vencido) e purga LGPD de inativos. Também serve healthcheck/liveness.
Serviços Externos (SMS Zenvia / WhatsApp Meta / Email / Google Calendar / Cloudflare Turnstile)	Sistemas de terceiros integrados. Zenvia entrega OTP/lembretes por SMS e posta status de entrega em webhook (IP allowlist + HMAC). WhatsApp (Meta Cloud API) recebe respostas via webhook (handshake hub.challenge + X-Hub-Signature-256) e captura nota NPS 0-10. Email (SMTP via Celery async) envia confirmações/cancelamentos/termos/promoções. Google Calendar (OAuth) conecta agenda do profissional (connect/callback/pull + feed ICS). Cloudflare Turnstile valida captcha anti-bot no booking/OTP.

3. Requisitos Funcionais

ID	Ator	Descriçao
RF01	Cliente (publico)	Agendar online escolhendo procedimento, profissional, data e horario disponiveis, fornecendo dados e aceitando os termos LGPD, com confirmacao do agendamento.
RF02	Cliente (publico)	Validar identidade por OTP via SMS (codigo de 6 digitos) quando for cliente existente no agendamento ou ao acessar dados, dentro da validade de 10 min e limite de tentativas/rate.
RF03	Sistema	Auto-aprovar agendamentos de cliente OTP-validado e sem bloqueio (entram como AGENDADO), encaminhando para PENDENTE apenas em execucao (sem OTP, conflito ou cliente bloqueado).
RF04	Cliente (publico)	Reagendar um atendimento ate 24h antes por link self-service valido por 48h, escolhendo novo horario disponivel.
RF05	Cliente (publico)	Cancelar um atendimento ate 24h antes via link magico de cancelamento.
RF06	Cliente (publico)	Confirmar presenca pelo link magico (ou WhatsApp), movendo o atendimento para CONFIRMADO.
RF07	Cliente (publico)	Acessar 'Meus agendamentos' (auto-servico) via login por email/telefone + OTP SMS, consultando e gerenciando seus proprios agendamentos sem senha.
RF08	Cliente (publico)	Entrar na lista de espera de um procedimento/profissional e, em caso de cancelamento, ser notificado e ter o slot reservado por 30 min.
RF09	Cliente (publico)	Preencher formulario de anamnese publica enviado por link (token) antes do atendimento.
RF10	Cliente (publico)	Responder pesquisa NPS (nota 0-10) e pesquisa pos-atendimento online via link com token.
RF11	Cliente (publico)	Solicitar export/consulta dos proprios dados (DSAR) em /lgpd/meus-dados mediante OTP DSAR, e descadastrar-se das comunicacoes via link de unsubscribe.
RF12	Recepcao	Aprovar ou rejeitar agendamentos PENDENTES (aprovar -> AGENDADO com e-mail; rejeitar -> CANCELADO) e agendar manualmente, inclusive para clientes bloqueados online.
RF13	Recepcao/Admin	Marcar atendimentos como REALIZADO, FALTOU, CANCELADO, disparando comissao, retorno obrigatorio, cashback e demais regras conforme a FSM.
RF14	Admin/Recepcao	Gerenciar clientes no painel: cadastro, edicao, busca (PII mascarada na API), soft-delete, consentimentos e contador de faltas/bloqueio.
RF15	Admin/Recepcao/Profissional	Visualizar e operar a agenda (FullCalendar), com horarios, bloqueios, execucoes, feriados e disponibilidade calculada por profissional.
RF16	Profissional	Acessar o portal/painel do profissional com sua agenda propria, prontuario e anotacoes de sessao, restrito aos seus proprios atendimentos (ownership anti-BOLA).
RF17	Admin/Profissional	Registrar e consultar prontuario, anamnese base e anotacoes clinicas de sessao por atendimento, com autor.
RF18	Admin/Recepcao	Gerenciar pacotes: vender pacote (sessoes/valor/expiracao), consumir 1 sessao por atendimento e acompanhar status (ATIVO/FINALIZADO/CANCELADO/EXPIRADO).
RF19	Admin	Criar e gerenciar promocoes (desconto 0-100% XOR preco fixo) e precos por vigencia sem sobreposicao, alem da habilitacao profissional-procedimento.
RF20	Admin	Configurar regras de comissao (RegraComissao por profissional/procedimento, percentual ou fixo) e acompanhar comissoes geradas e estornadas por atendimento.
RF21	Admin	Acompanhar e responder NPS coletado, visualizando notas e pesquisas pos-atendimento.
RF22	Admin/Sistema	Gerenciar notificacoes multicanal (WhatsApp, Email, SMS, Push PWA) com

4. Requisitos Nao-Funcionais

ID	Categoria	Descricao
RNF01	Seguranca	Content-Security-Policy com nonce por request, sem unsafe-inline em script-src/style-src (tests de regressao garantem). Permanece unsafe-inline apenas em script-src-attr/style-src-attr ate migrar handlers inline (Lote 3.5).
RNF02	Seguranca	Headers de seguranca: HSTS 1 ano com includeSubDomains e preload, X-Frame-Options DENY, X-Content-Type-Options nosniff, Referrer-Policy strict-origin-when-cross-origin, COOP/CORP e Permissions-Policy. SSL redirect e HTTPS forçado em producao.
RNF02b	Seguranca	2FA TOTP para staff (django-two-factor-auth) com QR, app autenticador e 10 backup codes; meta de tornar obrigatorio para ADMIN/staff que ve PII clinico.
RNF03	Seguranca	OTP cripto-seguro (secrets.randbelow), armazenado como hash SHA-256, nunca em texto plano nem logado; propositos isolados (AGENDAMENTO/LOGIN/DSAR); meta de endurecer com salt/pepper e hmac.compare_digest (constant-time).
RNF04	Seguranca	OTP anti-sequestro: cliente existente exige validacao OTP antes de agendar em seu nome; fluxos LOGIN e DSAR sao anti-enumeracao (resposta uniforme); meta de remover vazamento cliente_existente do fluxo de agendamento.
RNF05	Seguranca	Rate limiting em camadas: OTP reenvio >=60s, 3/h por telefone, 10/h por IP, 60/h global; HTTP 5-10/min por IP em endpoints sensiveis; django-axes 5 tentativas/cooloff 1h no login staff; DRF throttling 60/h anonimo e 1000/h user; Cloudflare Turnstile em forms publicos.
RNF06	Seguranca	Webhooks e cron jobs protegidos por token (CRON_TOKEN); CSRF token em forms e X-CSRFToken nos fetch com CSRF_TRUSTED_ORIGINS. (Meta de evolucao: assinatura HMAC nos tokens de contexto via TimestampSigner e nos webhooks recebidos.)
RNF07	Seguranca	Tokens gerados com secrets.token_urlsafe (cripto-seguros); cookies HTTPONLY + SameSite=Lax + Secure em prod; cycle_key apos login; sessao 8h e expira ao fechar o browser. Meta de TTL/revogacao nos tokens hoje infinitos (cancelamento, descadastro, ics).
RNF08	Seguranca	Gestao de secrets sem hardcode (env vars Railway), SECRET_KEY obrigatoria em prod, .gitignore/.dockerignore cobrindo segredos; limites de upload (5MB, 200 campos) e bleach/json_script contra XSS em conteudo controlado pelo usuario.
RNF09	LGPD	Consentimentos granulares (email_marketing, whatsapp_nps, whatsapp_confirmacao) com data e IP; aceite de termos versionados como evidencia legal; banner de cookies granular.
RNF10	LGPD	Soft-delete de entidades com PII (deletado_em) e job semanal de anonimizacao apos 30 dias; cliente inativo anonimizado em 5 anos; descadastro one-click por token de unsubscribe.
RNF11	LGPD	Direitos do titular implementados: DSAR/export JSON em /lgpd/meus-dados com challenge OTP, confirmacao/aceso, correcao pelo admin, portabilidade, revogacao de consent e eliminacao via soft-delete + purge job.
RNF12	LGPD	Politica de retencao por tipo de dado (OTP 24h, notificacao 12m, auditoria/financeiro 5 anos, prontuario/aceite ~20 anos CFM, inativo 5 anos) com jobs de expurgo; bases legais documentadas por tratamento; PII mascarada em logs, na API e filtrada no Sentry (send_default_pii=False).
RNF13	LGPD	Auditoria de acoes sensiveis em log_auditoria com autor, tabela, registro e IP (art. 37) e sanitizacao automatica de PII; procedimento de resposta a incidente com comunicacao a ANPD em ate 72h.
RNF14	Desempenho	Operacoes assincronas via Celery + Beat (notificacoes, workflows, jobs periodicos sempre async); cache via Redis com decorator utils.cache.cached; 87+ indices no banco (compostos para queries de dashboard). Deploy push-to-deploy em 2-3 min.
RNF15	Confiabilidade	Maquina de estados (FSM) do atendimento com transicoes validas e execucoes tipadas; anti double-booking por constraint EXCLUDE gist no banco; CHECK e UNIQUE constraints em campos enumerados e naturais; transaction.atomic em operacoes multi-tabela; EventBus dispara handlers apos transaction.on_commit.

5. Regras de Negocio

ID	Título	Descrição
RN01	Atores e papéis do staff	Usuario possui papel (ADMIN, PROFISSIONAL ou RECEPCAO) com CHECK no banco. ADMIN tem acesso total e equivale a is_staff; PROFISSIONAL gerencia agenda propria, prontuario e anotacoes; RECEPCAO gerencia agenda, clientes, confirmacoes e aprovacao de agendamentos. Cliente nunca e Usuario.
RN02	Identidade do cliente por telefone (passwordless)	Cliente nao tem conta nem senha; identifica-se pelo telefone canonico (apenas digitos, 10-11). CPF e email sao opcionais. Sem email, usa pseudo-email sms+<telefone>@shivazen.local. Unicidade garantida no banco via UNIQUE parcial entre clientes ativos (uniq_cliente_telefone_ativo). Cliente e criado por upsert (get_or_create) durante o agendamento.
RN03	Indicacao e base de cashback	O campo indicado_por aponta para o cliente que indicou, servindo de base para o cashback por indicacao.
RN04	Bloqueio online por faltas	Apos 3 faltas consecutivas (MAX_FALTAS_ANTES_BLOQUEIO) o cliente recebe bloqueado_online=True e nao consegue agendar online (recepcao ainda agenda manualmente). Comparecer zera o contador de faltas_consecutivas.
RN05	Soft-delete do cliente (LGPD)	Cliente nunca e apagado fisicamente em fluxo normal; usa-se deletado_em (soft-delete). Job semanal anonimiza apos 30 dias de soft-delete.
RN06	Fluxo do agendamento publico	O booking publico segue os passos: escolher procedimento -> profissional -> data/horario -> dados + OTP -> confirmar. Existe acesso direto por profissional (/agendar/<slug>/) e versao iframe (/embed/agendar/).
RN07	Antecedencia minima de agendamento	Antecedencia minima de 2h (ANTECEDENCIA_MINIMA_AGENDAMENTO; chave de registry agenda.antecedencia_min_horas). Configuravel por profissional via min_notice_horas (default 2, faixa 0-720h).
RN08	Antecedencia maxima de agendamento	Teto global de 90 dias (JANELA_MAXIMA_AGENDAMENTO; agenda.antecedencia_max_dias). Profissional pode definir valor menor (max_advance_dias); a janela efetiva e min(profissional, global). Resolve a divergencia historica 90d vs 60d.
RN09	Calculo de disponibilidade	Disponibilidade = horarios do profissional menos bloqueios, excocoes, feriados e slots ja ocupados, respeitando a duracao do procedimento e o buffer.
RN10	Anti double-booking no banco	Dois atendimentos do mesmo profissional nao podem se sobrepor; garantido por constraint EXCLUDE gist no banco (agenda.double_booking). Corrida na borda gera a mensagem 'horario acabou de ser reservado'.
RN11	Aceite LGPD obrigatorio antes de persistir	O aceite dos termos LGPD e obrigatorio antes de persistir o agendamento; registra AceiteTermo e os consents (registro de consents ocorre antes do create do Atendimento).
RN12	Maquina de estados do atendimento (FSM)	Estados: PENDENTE -> AGENDADO -> CONFIRMADO -> REALIZADO CANCELADO FALTOU REAGENDADO. Semantica fixada: AGENDADO = horario marcado; CONFIRMADO = cliente respondeu que vai (link/WhatsApp). Transicoes invalidas levantam TransicaoStatusInvalida.
RN13	Aprovacao / auto-aprovacao do agendamento	Decisao aprovada: auto-aprovar cliente OTP-validado e sem bloqueio, entrando direto como AGENDADO. PENDENTE fica reservado a excocoes (sem OTP, conflito ou cliente bloqueado); recepcao aprova (-> AGENDADO, dispara e-mail) ou rejeita (-> CANCELADO).
RN14	OTP hashado e com validade	Codigo OTP de 6 digitos gerado com secrets.randbelow (cripto-seguro), armazenado como hash SHA-256 (codigo_hash), nunca em texto plano nem logado. Validade de 10 minutos (TTL_OTP_MINUTOS / otp.validade_min). Gerar novo OTP invalida o anterior.
RN15	Tentativas e lockout do OTP	Maximo de tentativas por codigo leva a lockout (marca usado_em). Valor canonico fixado em 5 (otp.max_tentativas), resolvendo a divergencia 3 vs 5 entre constants.py e o model.
RN16	Rate limit do OTP em	Reenvio com cooldown de 60s; SMS limitado a 3/h por telefone

6. Casos de Uso

Diagrama de Casos de Uso (UML) – Area Publica (Cliente)

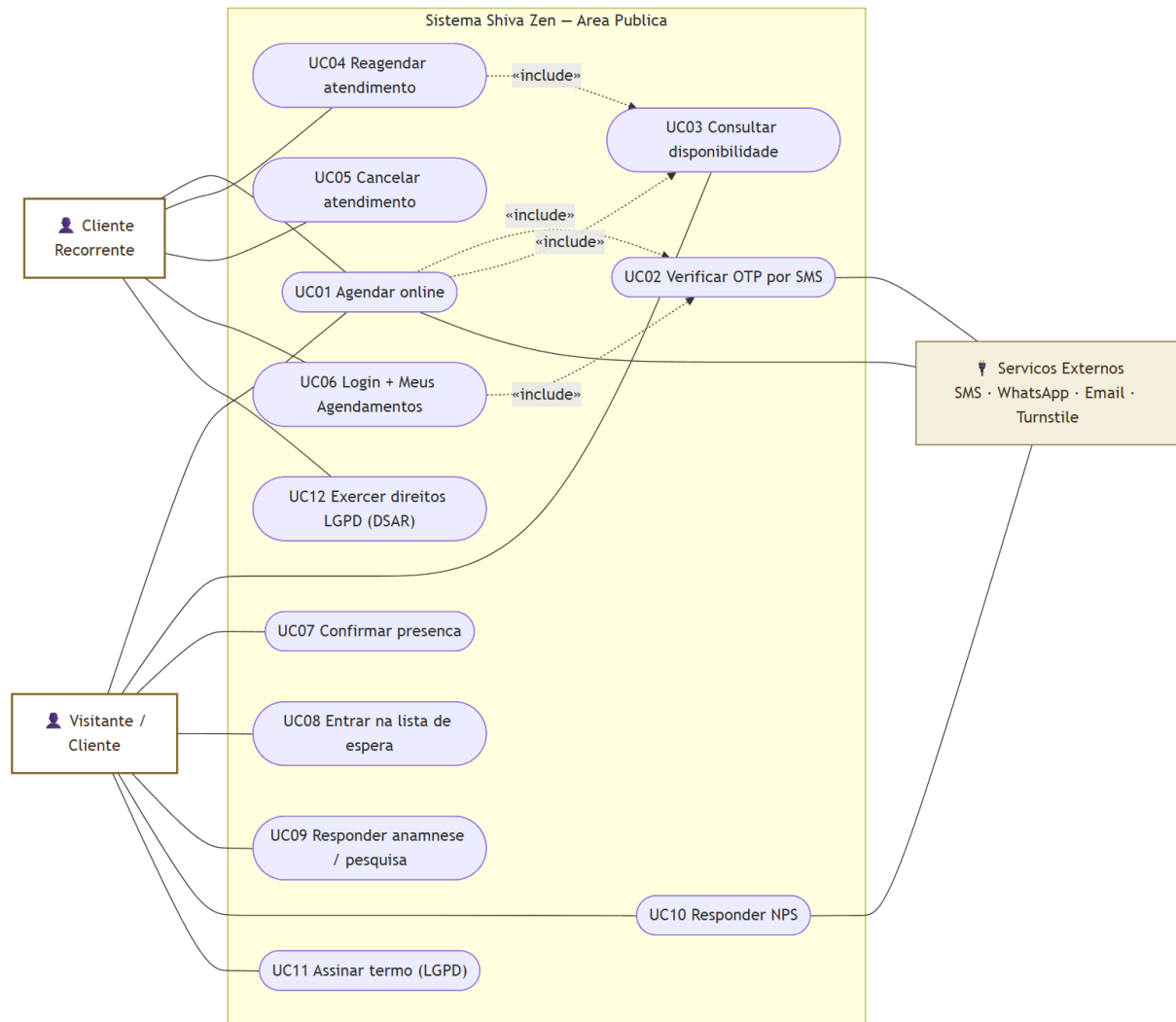


Figura 1 — Diagrama de Casos de Uso (UML): Area Publica (Cliente).

Diagrama de Casos de Uso (UML) – Area Interna (Staff, Profissional, Sistema)

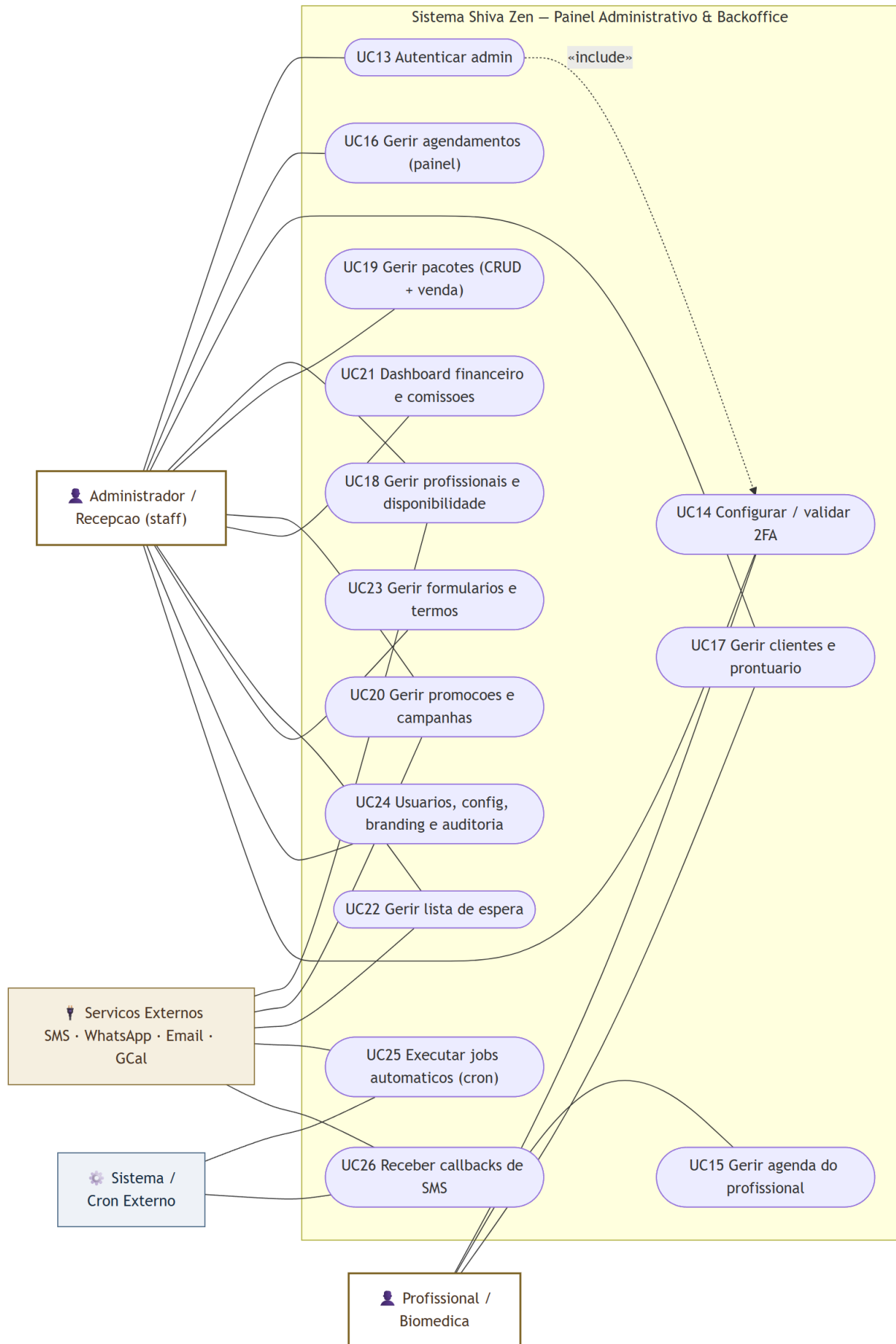


Figura 2 — Diagrama de Casos de Uso (UML): Area Interna (Staff, Profissional, Sistema).

6.1 Especificacao dos casos de uso

ID	Caso de Uso	Atores	Fluxo principal
UCO 1	Agendar online (sem login)	Visitante / Cliente (público, sem login), Cliente Recorrente (cadastro existente), Serviços Externos (SMS Zenvia / WhatsApp Meta / Email / Google Calendar / Cloudflare Turnstile)	Cliente escolhe procedimento, data/horário (3 steps) e preenche dados. confirmar_agendamento valida honeypot+Turnstile, idade ≥18, feriado, faz slot lock (cache.add 30s) + SELECT FOR UPDATE para evitar overbooking, get_or_create do Cliente por telefone com consentimentos LGPD, cria Atendimento status PENDENTE, grava respostas de anamnese inline e dispara emails async (termos pendentes ao cliente, aprovação ao profissional). Se cliente já existe (telefone OU email), exige OTP por SMS antes de prosseguir.
UCO 2	Verificar OTP por SMS	Cliente Recorrente (cadastro existente), Serviços Externos (SMS Zenvia / WhatsApp Meta / Email / Google Calendar / Cloudflare Turnstile)	solicitar_otp_agendamento gera código e envia por SMS (Zenvia) ao telefone (email opcional vira pseudo-email sms+digitos@ quando cadastro SMS-only); rate-limit 5/m e cooldown anti-abuso. verificar_otp_agendamento valida o código pelo otp_service e grava na sessão otp_agendamento_email + expiração 30min, devolvendo pré-fill do cliente. Anti-enumeração: resposta idêntica exista ou não o cadastro.
UCO 3	Consultar disponibilidade (horários e dias)	Visitante / Cliente (público, sem login)	Endpoints AJAX api_horarios_disponiveis (slots de 30min por data+procedimento, cruzando DisponibilidadeProfissional, Atendimentos ocupados e BloqueioAgenda, agrupados por horário/profissional) e api_dias_disponiveis (dias do mês com disponibilidade). buscar_procedimentos e buscar_horarios complementam. Rate-limit 30/m por IP.
UCO 4	Reagendar atendimento (público via token)	Cliente Recorrente (cadastro existente)	reagendar_agendamento resolve Atendimento por token_cancelamento, exige ≥24h de antecedência e status reagendável; em transação marca o antigo como REAGENDADO (sai da agenda antes para não colidir com a exclusion constraint), checa conflito com SELECT FOR UPDATE e cria novo Atendimento AGENDADO vinculado (reagendado_de) preservando valores. Redireciona para sucesso.
UCO 5	Cancelar agendamento (público via token)	Cliente Recorrente (cadastro existente)	cancelar_agendamento (POST JSON) busca Atendimento por token_cancelamento (anti-IDOR), bloqueia cancelamento de passados ou já cancelados, e em transação com SELECT FOR UPDATE seta status CANCELADO. Rate-limit 10/m por IP.

Shiva Zen — Requisitos v2.1 | Pag. 19 Shiva Zen — Requisitos v2.1 | Pag. 19 Shiva Zen — Requisitos v2.1 | Pag. 19 Shiva Zen —
Requisitos v2.1 | Pag. 19 Shiva Zen — Requisitos v2.1 | Pag. 19 Shiva Zen — Requisitos v2.1 | Pag. 19 Shiva Zen — Requisitos
v2.1 | Pag. 19 Shiva Zen — Requisitos v2.1 | Pag. 19 Shiva Zen — Requisitos v2.1 | Pag. 19 Shiva Zen — Requisitos v2.1 | Pag.
19 Shiva Zen — Requisitos v2.1 | Pag. 19 Shiva Zen — Requisitos v2.1 | Pag. 19 Shiva Zen — Requisitos v2.1 | Pag. 19

Diagrama de Classes (UML) – Núcleo de Agendamento

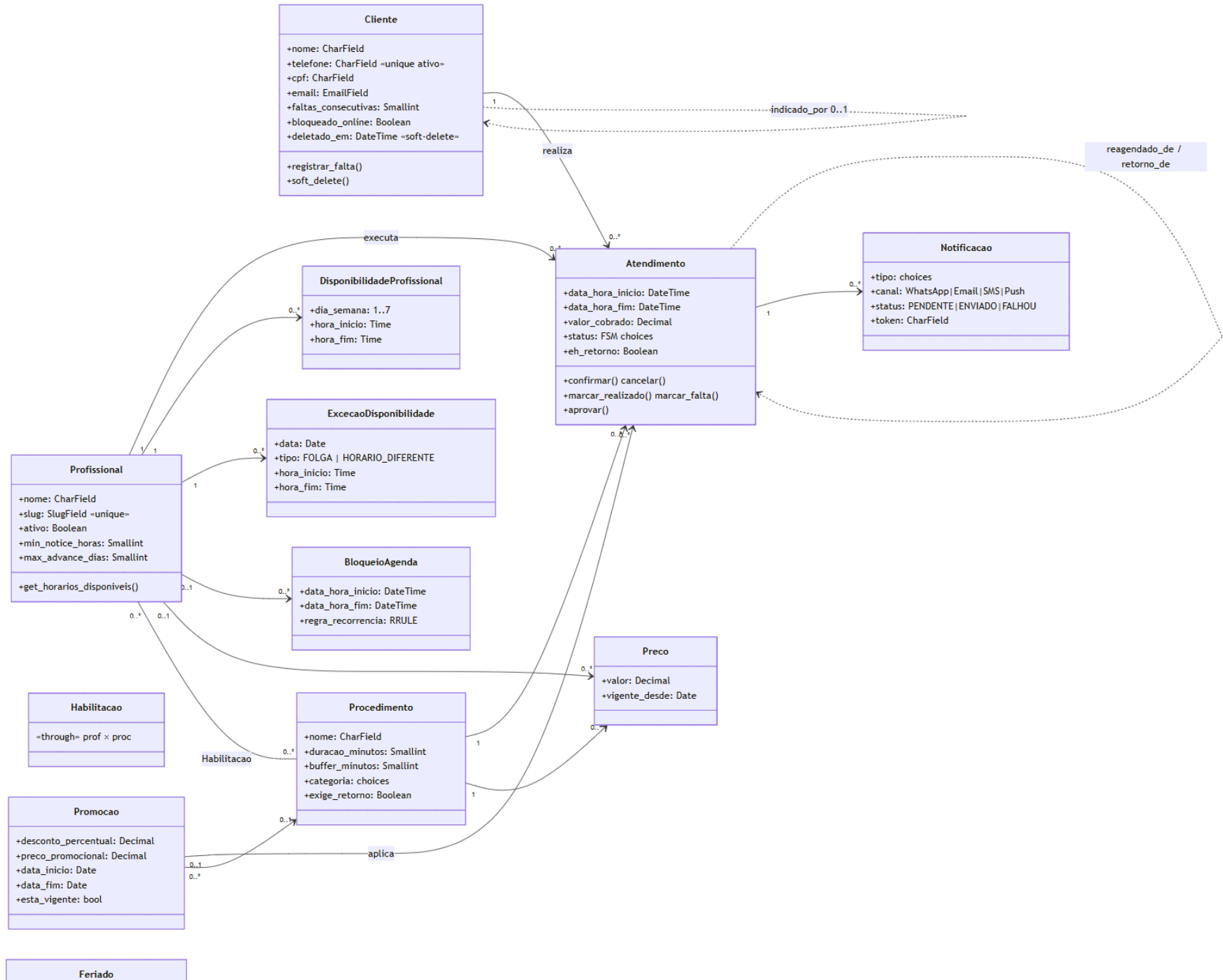


Figura 3 — Diagrama de Classes (UML): Nucleo de Agendamento.

Diagrama de Classes (UML) – Prontuario, Anamnese, NPS, Termos, Acesso e Auditoria

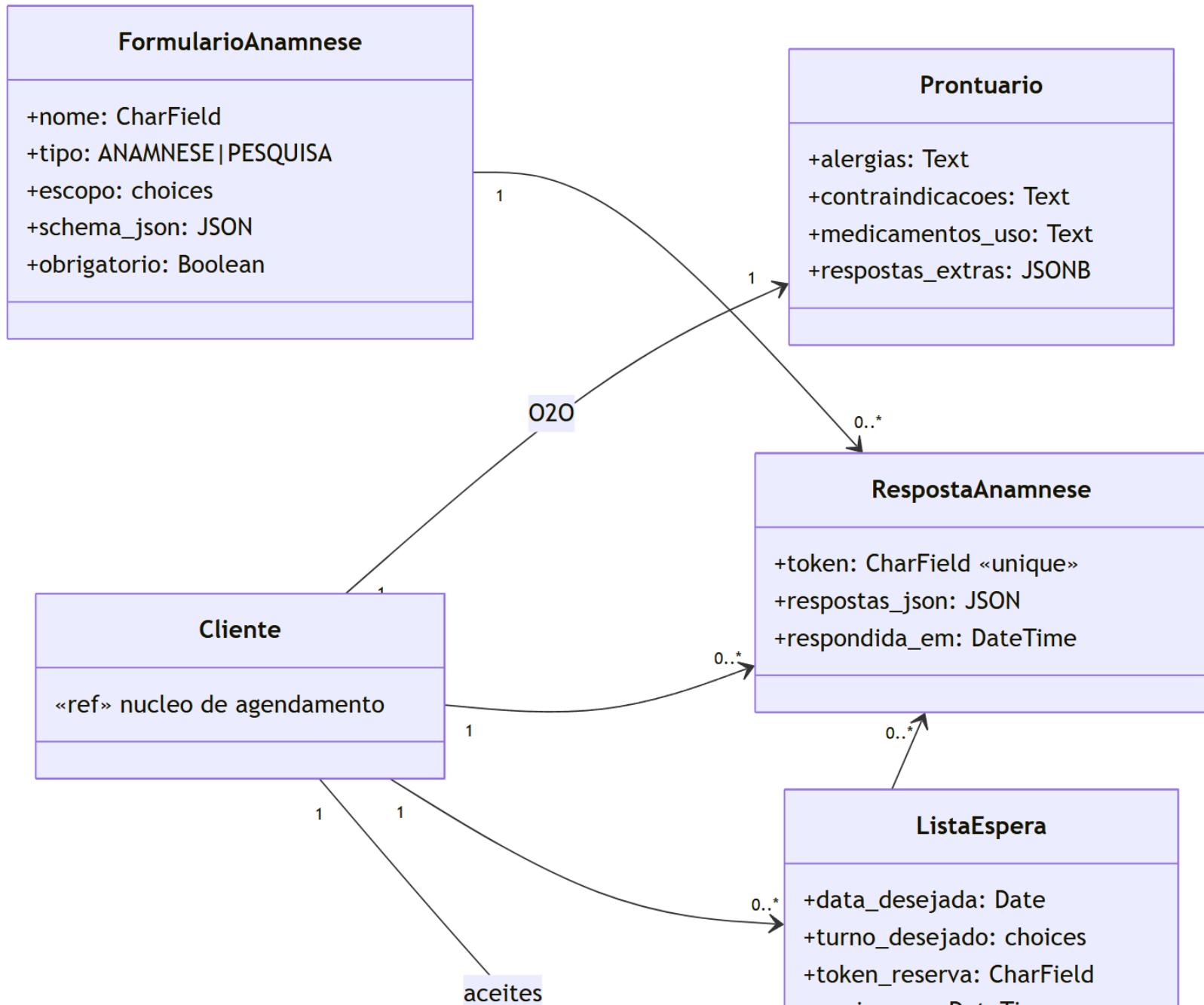


Figura 5 — Diagrama de Classes (UML): Prontuario, Anamnese, NPS, Termos, Acesso e Auditoria.

7.1 Dicionario de entidades

Entidade	Tabela	Descricao
Cliente	cliente	Cadastro de clientes da estetica com CRM, indicacao/fidelidade, consentimentos LGPD e soft-delete.
Profissional	profissional	Profissional que executa procedimentos, com regras de agendamento e integracao Google Calendar/ICS.
DisponibilidadeProfissional	disponibilidade_profissional	Regra de janela de atendimento semanal recorrente por dia da semana do profissional.
BloqueioAgenda	bloqueio_agenda	Bloqueio de agenda (unico ou recorrente via RRULE) de um profissional ou global (todos).
ExcecaoDisponibilidade	excecao_disponibilidade	Override pontual da disponibilidade semanal numa data: FOLGA (bloqueia dia) ou HORARIO_DIFERENTE.
Procedimento	procedimento	Servico/procedimento estetico com duracao, categoria, modalidade e regras de retorno obrigatorio.
Habilitacao	habilitacao	Tabela de ligacao (through) que habilita um profissional a executar um procedimento.
Preco	preco	Tabela de precos versionada por vigencia para um procedimento (preco-base ou especifico por profissional).
Promocao	promocao	Promocao com vigencia por datas aplicando desconto percentual OU preco fixo (XOR) a um procedimento.
Atendimento	atendimento	Agendamento/atendimento ligando cliente, profissional e procedimento, com maquina de estados (FSM) de status.
Notificacao	notificacao	Mensagem/notificacao (lembrete, confirmacao, NPS etc.) enviada por canal para um atendimento, com resposta.
Pacote	pacote	Pacote de servicos com preco fechado e validade em meses
ItemPacote	item_pacote	Linha de composicao do pacote: procedimento + qtd de sessoes incluidas
CompraPacote	compra_pacote	Compra de um pacote por um cliente, com status de consumo e data de expiracao
ConsumoSessao	consumo_sessao	Vincula um atendimento ao consumo de 1 sessao de uma compra de pacote
Prontuario	prontuario	Anamnese base permanente do cliente (1:1), com respostas extras em JSONB
AnotacaoSessao	anotacao_sessao	Observacoes clinicas especificas de cada atendimento, com autor
FormularioAnamnese	formulario_anamnese	Template configuravel de questionario (ANAMNESE pre ou PESQUISA pos), campos em schema_json com escopo
RespostaAnamnese	resposta_anamnese	Resposta de cliente a um formulario via link publico tokenizado, vinculada a atendimento
AvaliacaoNPS	avaliacao_nps	Avaliacao NPS (nota 0-10) vinculada 1:1 a um atendimento, com flag de alerta para nota baixa
ListaEspera	lista_espera	Fila de espera de cliente por procedimento/data/turno, com token de reserva e notificacao
LogAuditoria	log_auditoria	Trilha de auditoria de acoes de usuario sobre tabela/registro, com IP e detalhes JSON
Feriado	feriado	Datas bloqueadas para agendamento (feriado nacional/local ou recesso da clinica)
Configuracao	configuracao	Chave-valor de configuracao do sistema (ex.: schema das perguntas do prontuario)

8. Diagrama de Estados — Atendimento (FSM)

O status do atendimento e governado por uma maquina de estados finitos; transicoes invalidas levantam excecao tipada (Atendimento.TransicaoInvalida). Estado inicial: PENDENTE.

Diagrama de Estados (UML) – Ciclo de vida do Atendimento

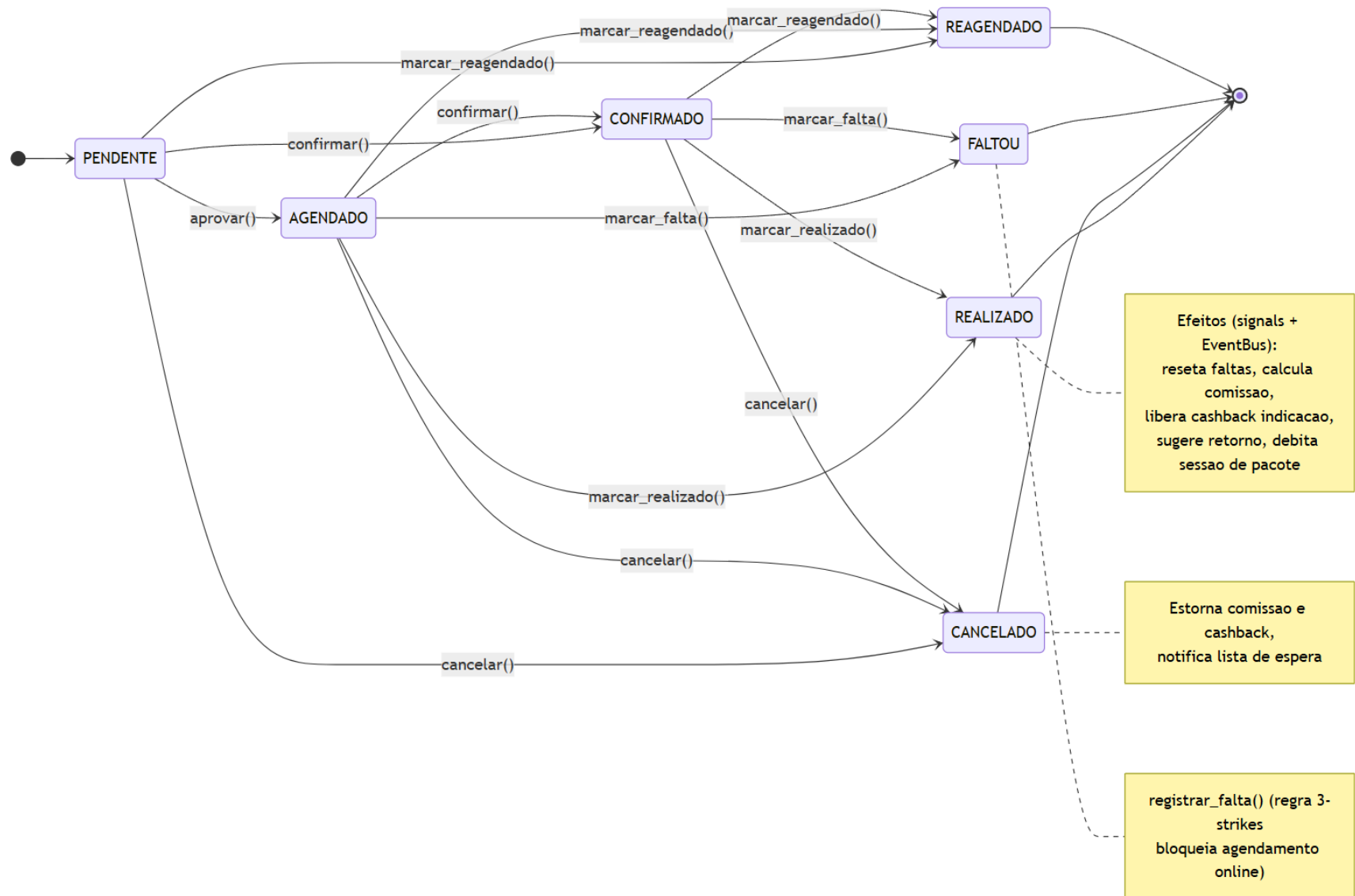


Figura 6 — Diagrama de Estados (UML): ciclo de vida do Atendimento.

8.1 Tabela de transicoes

Estado origem	Estado destino	Acao / gatilho
PENDENTE	AGENDADO	aprovar(by_user=None)
PENDENTE	CONFIRMADO	confirmar(by_user=None)
PENDENTE	CANCELADO	cancelar(motivo="", by_user=None)
PENDENTE	REAGENDADO	marcar_reagendado(by_user=None)
AGENDADO	CONFIRMADO	confirmar(by_user=None)
AGENDADO	REALIZADO	marcar_realizado(by_user=None)
AGENDADO	CANCELADO	cancelar(motivo="", by_user=None)
AGENDADO	FALTOU	marcar_falta(by_user=None)
AGENDADO	REAGENDADO	marcar_reagendado(by_user=None)
CONFIRMADO	REALIZADO	marcar_realizado(by_user=None)
CONFIRMADO	CANCELADO	cancelar(motivo="", by_user=None)
CONFIRMADO	FALTOU	marcar_falta(by_user=None)
CONFIRMADO	REAGENDADO	marcar_reagendado(by_user=None)

8.2 Efeitos colaterais reativos (signals / EventBus)

- **TRANSICAO** (todas): `_transicionar` valida em **TRANSICOES** e grava status via `save(update_fields=['status','atualizado_em'])` dentro de `transaction.atomic()`; cria `LogAuditoria` 'Atendimento' `<pk>`: `<de>` -> `<para>`' (savepoint best-effort, falha de DB nao desfaz a transicao).
- **CRIACAO** (`post_save created=True, signals.py`): `on_commit` dispara push ao profissional ('Novo agendamento') e sync Google Calendar (`push_atendimento`) se `gcal_disponivel()` e `profissional.gcal_refresh_token`. Best-effort.
- `confirmar()` -> **CONFIRMADO**: publica `AtendimentoConfirmado(confirmado_por_id)`; handler `domain/handlers.py log_confirmacao` apenas faz logging estruturado.
- `cancelar()` -> **CANCELADO**: publica `AtendimentoCancelado(motivo, cancelado_por_cliente=(by_user is None))`. `Handlers EventBus`: (1) `comissao_service._on_cancelado_estorna_comissao` -> `ComissaoService.estornar_comissao (MovimentoComissao PENDENTE/PAGA -> ESTORNADA)`; (2) `fidelidade_service._on_cancelado_estorna_cashback` -> `estornar_cashback_de_atendimento (motivo 'atendimento cancelado')`; (3) `lista_espera_service._on_cancelado_notifica_espera` -> `ListaEsperaService.notificar_compatíveis`.
- `marcar_realizado()` -> **REALIZADO**: publica `AtendimentoRealizado(cliente_id, profissional_id)`. `Handlers EventBus`: (1) `comissao_service._on_realizado_calcula_comissao` -> `ComissaoService.calcular_comissao`; (2) `fidelidade_service._on_realizado_libera_cashback` -> `liberar_cashback_indicacao (credita indicador)`; (3) `retorno_service._on_realizado_sugere_retorno` -> `RetornoService.sugerir_retorno (F-RET)`.
- `marcar_realizado()` -> **REALIZADO** (`signals.py post_save, status_atual=='REALIZADO'`): `cliente.resetar_faltas()` (zera faltas consecutivas) + debita sessao de pacote ativo (`CompraPacote select_for_update, ConsumoSessao.create, pc.verificar_finalizacao`) salvo se ja houver sessao_pacote_vinculada; pacotes expirados -> status **EXPIRADO**.
- `marcar_falta()` -> **FALTOU**: publica `AtendimentoFaltou(cliente_id)` (sem handler `EventBus` registrado). `signals.py: status_atual=='FALTOU'` vindo de **PENDENTE/AGENDADO/CONFIRMADO** -> `cliente.registrar_falta()` (3-strike).
- `marcar_reagendado()` -> **REAGENDADO**: **NAO** publica `DomainEvent` (so transiciona+audita).
- **LIBERA VAGA** (`signals.py`): transicao para **CANCELADO/FALTOU/REAGENDADO** vindo de **PENDENTE/AGENDADO/CONFIRMADO** -> `job_notificar_fila_espera.delay(procedimento_id, data_livre)` (Celery, notifica fila de espera).
- **NPS**: **NAO** disparado por handler de transicao. Celery `job_pesquisa_satisfacao_24h` varre `Atendimento status='REALIZADO', data_hora_fim<=now-1d`, sem `AvaliacaoNPS`, `cliente.consent_whatsapp_nps=True`, sem `Notificacao NPS PENDENTE/ENVIADO` -> envia NPS por WhatsApp. `job_alerta_detrator_nps` alerta admin por email quando nota NPS<=6.

- Shiva Zen — Requisitos v2.1 | Pag. 33 Shiva Zen — Requisitos v2.1 | Pag. 33 Shiva Zen — Requisitos v2.1 | Pag. 33 Shiva Zen —
Requisitos v2.1 | Pag. 33 Shiva Zen — Requisitos v2.1 | Pag. 33 Shiva Zen — Requisitos v2.1 | Pag. 33 Shiva Zen — Requisitos
v2.1 | Pag. 33 Shiva Zen — Requisitos v2.1 | Pag. 33 Shiva Zen — Requisitos v2.1 | Pag. 33 Shiva Zen — Requisitos v2.1 | Pag.
33 Shiva Zen — Requisitos v2.1 | Pag. 33 Shiva Zen — Requisitos v2.1 | Pag. 33 Shiva Zen — Requisitos v2.1 | Pag. 33

9. Diagrama de Sequencia — Agendamento com OTP

Fluxo do agendamento publico com o gate de verificacao OTP por SMS (anti-sequestro de cadastro), incluindo controle de concorrencia de slot e criacao transacional do atendimento.

Diagrama de Sequencia (UML) – Agendamento publico com OTP anti-sequestro

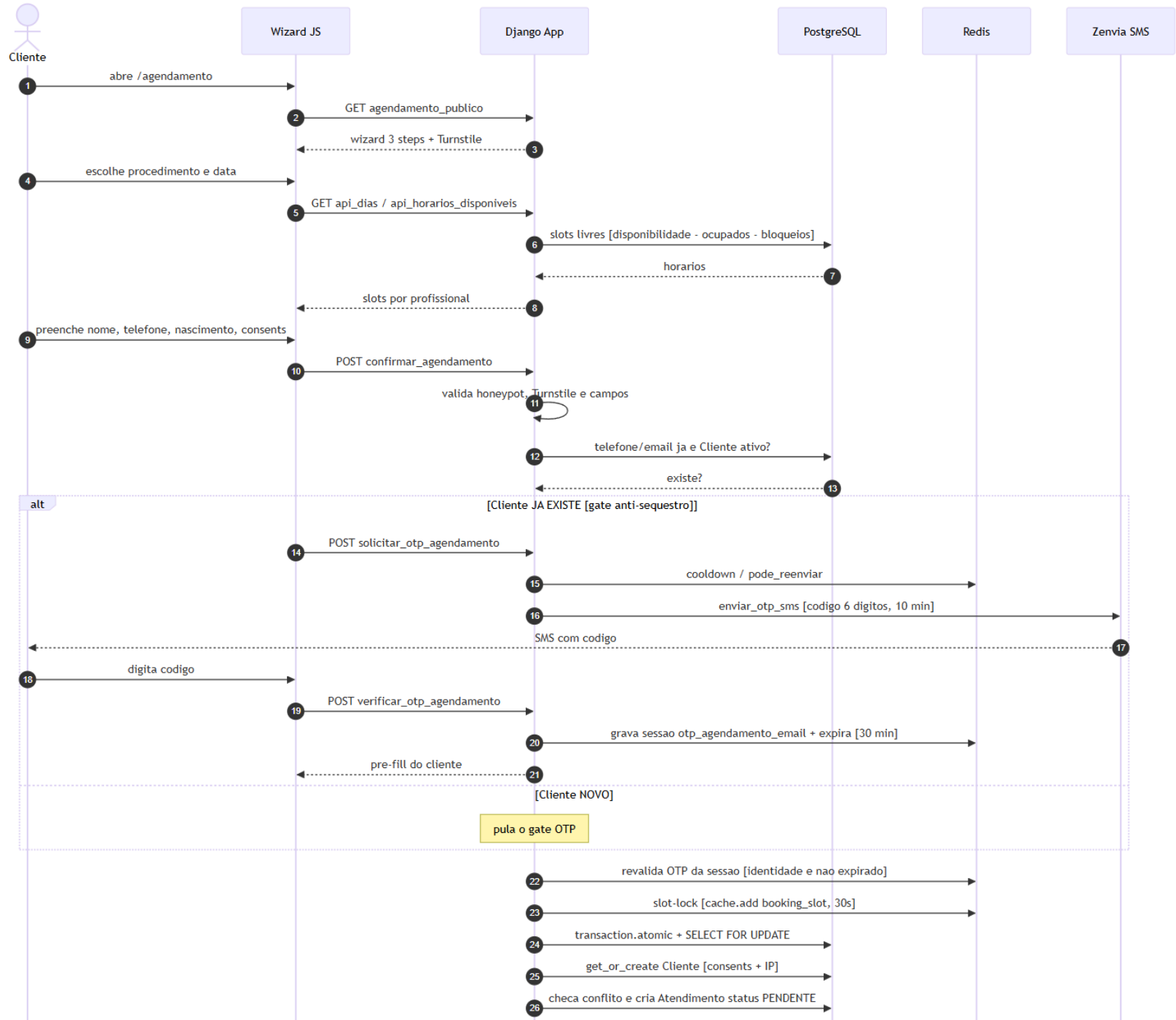
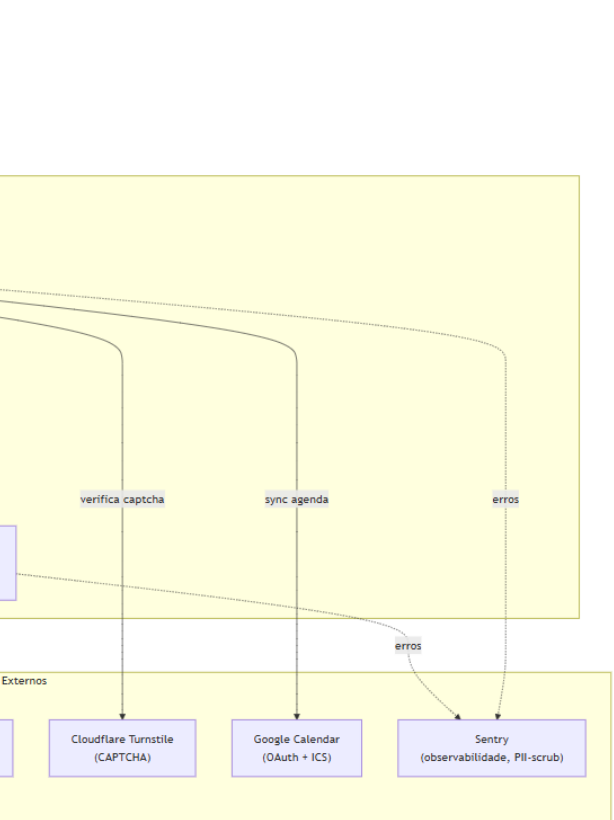


Figura 7 — Diagrama de Sequencia (UML): agendamento publico com OTP.

9.1 Passo a passo

1. 1. Cliente -> abre /agendamento (GET agendamento_publico, rate-limit 60/h por IP) -> Django App renderiza o wizard de 3 steps (procedimento, data/horario, dados) com procedimentos+precos, formularios de anamnese e site key do Turnstile.
2. 2. Cliente -> escolhe procedimento e data -> wizard.js consulta api_dias_disponiveis e api_horarios_disponiveis (booking_api.py, 30/m por IP) -> Django App/PostgreSQL retorna slots livres por profissional (sem overbooking).
3. 3. Cliente -> preenche nome, telefone, data de nascimento, email (opcional) e consents no step de confirmacao.
4. 4. Cliente (JS) -> ao submeter, POST em confirmar_agendamento (booking_public.py, rate-limit 10/m por IP) -> Django App valida honeypot 'website' (bot -> redireciona) e, se turnstile_enabled, valida o token Turnstile via Cloudflare; normaliza o telefone e checa campos obrigatorios (booking_public.py:116-141).
5. 5. Django App -> GATE ANTI-SEQUESTRO: verifica se o TELEFONE (ou, se informado, o EMAIL) ja pertence a um Cliente ativo (Cliente.objects.filter(telefone=...).exists() / email__iexact) — identidade real do booking e o telefone (booking_public.py:142-149).
6. 6. Se o cliente JA EXISTE -> antes de qualquer escrita o JS precisa ter obtido um OTP: POST em solicitar_otp_agendamento (booking_otp.py, 5/m por IP) -> valida Turnstile, exige telefone, resolve a identidade (email real ou pseudo-email sms+<digitos> viaCodigoOtp.email_para_telefone) e chama otp_service.solicitar_otp(proposito=AGENDAMENTO, canal=SMS).
7. 7. otp_service.solicitar_otp -> geraCodigoOtp (com cooldown/pode_reenviar anti-abuso) -> OTPService.enviar_codigo -> utils/sms.enviar_otp_sms -> Zenvia SMS entrega o codigo (valido 10 min) ao telefone do cliente; resposta e identica exista ou nao o cadastro (anti-enumeracao).
8. 8. Cliente -> digita o codigo -> POST em verificar_otp_agendamento (booking_otp.py, 10/m por IP) -> otp_service.verificar_otp valida o codigo; em sucesso grava na SESSAO request.session['otp_agendamento_email'] e request.session['otp_agendamento_expira'] (TTL 30 min) e devolve pre-fill do cliente (booking_otp.py:81-92).
9. 9. Django App (no POST de confirmar_agendamento) -> REVALIDA o OTP da sessao: o otp_agendamento_email deve estar entre as identidades aceitas (pseudo-email do telefone e/ou email do form) e otp_agendamento_expira deve ser futuro; se invalido/expirado -> erro 'Confirme com o codigo SMS...' e redirect (booking_public.py:151-171). Cliente novo (sem cadastro) pula o gate OTP.
10. 10. Django App -> valida idade >= 18 (data_nascimento), valida data/hora futura, bloqueio de feriado, e adquire slot-lock no cache (cache.add booking_slot:<prof>:<datetime>, 30s) para evitar corrida (booking_public.py:173-205).
11. 11. Django App -> em transaction.atomic + SELECT FOR UPDATE: get_or_create do Cliente por telefone (atualiza nome/email/consents com data+IP), checa conflito de horario do profissional e, se livre, cria o Atendimento com status PENDENTE (valor_cobrado calculado por preco_para); persiste respostas de anamnese e, se houver termos LGPD/procedimento nao assinados, cria Notificacao com token de termo (booking_public.py:207-336).
12. 12. Django App -> dispara emails assincronos via Celery (send_email_async): termos pendentes ao cliente (se email) e aviso de novo agendamento PENDENTE ao profissional (com links aprovar/rejeitar). Worker/SMTP entregam fora do request.
13. 13. Django App -> grava request.session['agendamento_sucesso'], limpa as chaves de OTP da sessao (otp_agendamento_email/expira) e redireciona para agendamento_sucesso, que exhibe a confirmacao (atendimento aguardando aprovacao da clinica) com botao WhatsApp (booking_public.py:358-408).

Shiva Zen — Requisitos v2.1 | Pag. 38 Shiva Zen — Requisitos v2.1 | Pag. 38 Shiva Zen — Requisitos v2.1 | Pag. 38 Shiva Zen —
Requisitos v2.1 | Pag. 38 Shiva Zen — Requisitos v2.1 | Pag. 38 Shiva Zen — Requisitos v2.1 | Pag. 38 Shiva Zen — Requisitos
v2.1 | Pag. 38 Shiva Zen — Requisitos v2.1 | Pag. 38 Shiva Zen — Requisitos v2.1 | Pag. 38 Shiva Zen — Requisitos v2.1 | Pag.
38 Shiva Zen — Requisitos v2.1 | Pag. 38 Shiva Zen — Requisitos v2.1 | Pag. 38 Shiva Zen — Requisitos v2.1 | Pag. 38



10.1 Componentes

Componente	Papel
Django App (web / gunicorn)	Aplicacao monolitica Django (app unico 'aranha_estetica' + apps de auth/otp/2fa/axes/DRF/spectacular). Servida por gunicorn (Procfile: 4 workers x 2 threads; railway.json: 1 worker x 4 threads, timeout 120). Roteia o booking publico (views/booking_public.py, booking_api.py, booking_otp.py), API REST (DRF SessionAuth + throttling anon 60/h, user 1000/h), admin com 2FA, healthcheck /healthz/. WSGI = clinica.wsgi.
Celery Worker	Processa tasks assincronas de aranha_estetica/tasks.py: envio de email transacional/marketing fire-and-forget (send_email_async), lembrete D-1 WhatsApp, NPS 24h, alerta detrator NPS, fila de espera, pacotes expirando/expirar, aniversario, limpeza de status de atendimentos, purga LGPD. Procfile: celery -A clinica worker --concurrency=2. Todos os jobs com max_retries=3 e retry com backoff.
Celery Beat (scheduler)	Agendador de jobs periodicos via CELERY_BEAT_SCHEDULE (crontab): lembretes 08h, NPS 10h, alerta detrator 10h30, verificar pacotes 07h, expirar pacotes 00h30, limpeza status 23h, aniversario 09h, purga LGPD domingo 03h. Procfile: celery -A clinica beat.
Redis (broker + cache + sessao)	Tripla papel. (1) Broker e result backend do Celery (CELERY_BROKER_URL / CELERY_RESULT_BACKEND, default redis://localhost:6379/0). (2) Cache default via django_redis quando REDIS_URL definido, sustentando rate-limit/django-axes/slot-lock de booking e os contadores de quota de SMS (sms_rl:*). (3) Backend de sessao (SESSION_ENGINE=cache) — onde vivem as chaves otp_agendamento_email/expira. Sem REDIS_URL cai para LocMemCache + sessao em DB (com warning em prod).
PostgreSQL	Banco de dados primario em prod via DATABASE_URL (dj_database_url, conn_max_age=600, health checks, ssl_require fora de DEBUG; Railway gerenciado). Guarda Cliente, Atendimento (com constraint EXCLUDE excl_atendimento_sobreposicao anti-overbooking e FSM de status), Procedimento, Profissional, Notificacao,CodigoOtp, termos/anamnese etc. Em dev usa SQLite (db_dev.sqlite3); testes sempre SQLite.
Vite / Frontend (django-vite)	Bundler do frontend (Vite 6) integrado via django_vite. Em DEBUG roda dev server (dev_mode, hot reload); em prod le manifest.json em aranha_estetica/static/dist/ com static_url_prefix 'dist'. Templates renderizados com django-cotton (componentes) + django-staticfiles. JS do wizard de agendamento (static/js/wizard.js) consome os endpoints AJAX de horarios/dias/OTP.
WhiteNoise (static serving)	Middleware que serve assets estaticos comprimidos em prod (CompressedManifestStaticFilesStorage) sem servidor de arquivos dedicado; collectstatic roda no startCommand do railway.json e GZip middleware acompanha.
PWA / Service Worker (web push)	Camada PWA com service worker e web push (static/js/webpush.js, base_v2.html do painel). Registra SW e gerencia push notifications para o painel administrativo; CSP do middleware precisa liberar os recursos do worker.
Middleware de seguranca (CSP + headers + 2FA + axes)	aranha_estetica.middleware: ContentSecurityPolicyMiddleware (injeta CSP com nonce, exposto via context_processor csp_nonce), SecurityHeadersMiddleware (headers extras), Enforce2FAMiddleware (forca 2FA no admin/painel) e AxesMiddleware (lockout por IP+username, 5 falhas, cooloff 1h). Atras de Cloudflare/Railway respeita X-Forwarded-For/Proto.

10.2 Integracoes externas

Servico	Uso
Zenvia SMS (REST API v2)	Canal PRIMARIO e exclusivo de OTP (verificacao anti-sequestro do agendamento e login 'Meus Agendamentos'). utils/sms.py: POST em https://api.zenvia.com/v2/channels/sms/messages com header X-API-TOKEN (ZENVIA_API_TOKEN) e remetente ZENVIA_FROM; telefone normalizado para E.164 (55...). Tem rate-limit anti-abuso por telefone (3/h), por IP (10/h) e global (60/h) via cache. retrv exponencial em 5xx/timeout. e modo dev (DEBUG ou

Historico de Versoes

Versao	Data	Descricao
2.1	21/06/2026	Documento de requisitos + modelagem UML gerado a partir do codigo (pos-auditoria SWE de backend e migracao de frontend base_v2).